Patient Risk Trend Analyzer

ML.NET web app that turns patient vital-sign time series into trend features, a disease-risk prediction and a plain-language insight

Stack	C# / .NET 9, ASP.NET Core minimal API, ML.NET (LbfgsLogisticRegression), plain HTML/JS front end
Input	JSON file of timestamped readings: blood pressure, glucose, SpO2, weight
Output	Risk label, probability, risk band, certainty, contributors, alerts, trend lines, recommendation
Training data	2,000 synthetic, rule-labelled samples generated at startup
Status	Working prototype with documented data-quality gaps

DISCLAIMER: This is an educational prototype. It is not a medical device and must not be used for diagnosis or treatment decisions. The model is trained on synthetic data and its outputs have no clinical validity.

1. Overview

People who track vitals at home produce small, irregular time series. A single reading says little; the direction of change matters more. This project reduces a patient's readings to eight numeric features, feeds them to an ML.NET logistic-regression classifier and presents the result as a risk band, the reasons behind it, and a suggested next step.

The application is a single ASP.NET Core process. The model trains once at startup, a single POST endpoint serves predictions, and one static HTML page is the user interface. There is no database and no external service.

2. Project Structure

File	Role
Program.cs	Builds the web host, registers RiskPredictionService as a singleton, forces model training at startup, serves wwwroot, and maps POST /api/predict (returns 400 with an error message on validation failure).
Models.cs	VitalType flags enum, JSON input classes, VitalReading record, PatientTrendFeatures (8 features + label), RiskPrediction, EndUserInsight, PredictionResult.
Services.cs	TrendFeatureBuilder, SyntheticTrainingDataFactory, EndUserInsightEngine, RecommendationEngine.
RiskPredictionService.cs	Trains and evaluates the model in the constructor; Validate() and RunFullPrediction() for each request.
sample-patient-input.json	Example input for the upload button.
wwwroot/index.html	Entire UI (HTML + CSS + JS). Not included in the files reviewed for this report.

3. Architecture and Request Flow

```
Browser (index.html)
   |  POST /api/predict   (PatientInputFile JSON)
   v
Program.cs  --->  RiskPredictionService.RunFullPrediction()
                    1. Combine TrackedVitals into a VitalType bitmask
                    2. Validate()            (name, >=1 vital, >=5 readings, non-null)
                    3. Map JSON readings -> VitalReading records (Day = index + 1)
                    4. TrendFeatureBuilder.Build()      -> 8 features
                    5. ML.NET PredictionEngine          -> label, probability, score
                    6. EndUserInsightEngine + RecommendationEngine
   v
PredictionResult JSON  ->  rendered by the page
```

A lock around prediction-engine creation keeps concurrent requests safe. The PredictionEngine is created on every request, which is simple but not the most efficient pattern (see Section 10).

4. Feature Engineering

Eight features are derived from the readings. Vitals the patient is not tracking are replaced by fixed neutral defaults so the model still receives a full vector.

Feature	Computation	Default if untracked
AvgSystolic	Mean of Systolic over all readings	120
AvgDiastolic	Mean of Diastolic over all readings	80
AvgGlucose	Mean of Glucose	100
AvgOxygen	Mean of Oxygen (SpO2 %)	98
WeightChange	last.WeightKg - first.WeightKg	0
SystolicTrend	(last.Systolic - first.Systolic) / readings.Count	0
GlucoseTrend	(last.Glucose - first.Glucose) / readings.Count	0
OxygenTrend	(last.Oxygen - first.Oxygen) / readings.Count	0

The trend features use only the first and last reading, so spikes or oscillations in between do not affect them. The averages use every reading.

5. Model and Training

- **Trainer:** ML.NET `LbfgsLogisticRegression`, label column `Label`, `MLContext(seed: 1)`.
- **Data:** 2,000 samples from `SyntheticTrainingDataFactory` (random seed 7), split 80/20 with `TrainTestSplit`.
- **Evaluation:** accuracy and AUC on the 20% test set, computed once at startup and returned with every prediction.

Synthetic features are drawn uniformly from these ranges:

Feature	Range	Feature	Range
Avg systolic	95 - 180	Systolic trend	-1.5 - 2.5
Avg diastolic	60 - 115	Glucose trend	-2.5 - 4.0
Avg glucose	70 - 280	Oxygen trend	-0.8 - 0.4
Avg oxygen	85 - 100	Weight change	-8 - 10

The label comes from a threshold score. Each condition adds points, random noise of +/-0.4 is added, and a sample is labelled at-risk when the total is at least 2.1:

Condition	Points	Condition	Points
Avg systolic > 140	1.2	Systolic trend > 1.2	0.7
Avg diastolic > 90	0.8	Glucose trend > 1.8	0.9
Avg glucose > 170	1.4	Oxygen trend < -0.3	0.9
Avg oxygen < 93	1.8	Weight change > 5	0.5

Consequence: the labels are a function of the same features the model sees, so it can only rediscover this rule. High accuracy and AUC on the test split reflect rule recovery, not clinical skill.

6. Insight and Recommendation Engines

Risk band and **recommendation** are driven by the predicted probability:

Probability	Risk band	Recommendation
80% or more	Very High	Consult a physician within 24 hours; consider urgent care if symptoms exist.
60% - 79%	High	Schedule a doctor consultation this week and repeat vitals daily.
35% - 59%	Moderate	Improve diet/activity, monitor 1-2 weeks, discuss with primary doctor.
Below 35%	Low	Continue tracking and routine checkups; if tracked SpO2 average is below 92, advise seeking clinician advice soon.

Note the recommendation tiers are labelled differently from the risk bands: 60-79% is called "High" in the band but "Moderate-high" in the recommendation text.

Certainty measures distance from the 0.5 decision boundary: 0.35 or more is High, 0.20 or more is Medium, otherwise Low.

Contributors and alerts are rule-based text generated from the features, for example average BP at or above 140/90, average glucose at or above 140 mg/dL, average SpO2 below 94%, systolic trend above +1.0, oxygen trend below -0.2, or weight gain above 3 kg. These thresholds are independent of the model, so the explanation text and the model's prediction can disagree.

7. Input Format and Validation

```
{
  "PatientName": "Sample Patient",
  "TrackedVitals": ["BloodPressure", "BloodGlucose", "Oxygen", "Weight"],
  "Readings": [
    { "RecordedAt": "2026-05-25T18:38:00", "Systolic": 118, "Diastolic": 76,
      "Glucose": 105, "Oxygen": 97, "WeightKg": 72.4 }
  ]
}
```

`Validate()` currently rejects a request when: the patient name is blank; no vital is tracked; fewer than 5 readings are supplied; or a tracked vital is *null* in any reading. It does not check values. This is the source of the main findings below.

8. Audit of the Supplied Test File

The supplied `sample-patient-input.json` contains 13 readings from 2026-05-25 to 2026-06-10, tracking all four vitals. Inspecting each column and tracing it through `TrendFeatureBuilder` gives:

Field	Observation
Systolic / Diastolic	11 of 13 readings plausible (91-117 / 59-75). Reading #2 is 0 / 0, which is not a real measurement.
Glucose	0 in all 13 readings (never recorded).
Oxygen	0 in all 13 readings (never recorded).
Weight (kg)	0 in 12 readings; one reading of 365 kg (reading #2), physiologically implausible.

Features the builder computes from this file (hand-calculated from the JSON; the model's output was not run for this report):

Feature	Value	Why it is wrong
AvgSystolic	96.2	Includes the 0 from reading #2; true mean of the 12 real values is about 104.2.
AvgDiastolic	61.1	Same 0 included; true mean of the 12 real values is about 66.2.
AvgGlucose	0	Unrecorded vital averaged as zero.
AvgOxygen	0	Unrecorded vital averaged as zero; training range was 85-100.
WeightChange	0	First and last weights are both 0; the 365 kg outlier is masked by luck.
SystolicTrend	+0.62	(101 - 93) / 13.
GlucoseTrend / OxygenTrend	0 / 0	Flat zeros, not real trends.

An average oxygen of 0 and average glucose of 0 lie far outside the ranges the model was trained on. The classifier will extrapolate, and the insight engine will print alerts such as "Average SpO2 is low at 0.0%". A missing measurement is thus presented as a medical emergency.

9. Findings

9.1 Defects

- **Zero is accepted as a measurement.** Validation checks only for null. Zero values (including 0/0 blood pressure) pass and are averaged into the features.
- **No plausibility ranges.** A 365 kg weight, SpO2 of 0 or glucose of 0 all pass validation.
- **"Per day" trends are really per reading.** Trend features divide by the number of readings, and the Day field (index + 1) is never used, yet the insight text labels the units mmHg/day, mg/dL/day and %/day. In the test file the 13 readings span 16 days, so even a correct per-reading slope would differ from a per-day slope. Dividing by (count - 1) or by elapsed days from RecordedAt would be more accurate.
- **Synthetic, self-referential labels.** See Section 5.
- **Insight text is decoupled from the model.** Contributors and alerts use hard-coded thresholds that differ from the labelling thresholds, so they can disagree with the predicted risk.

9.2 Documentation issues

- README refers to `PatientVitalsPrediction.Simple.csproj`, but the uploaded file is named `PatientVitalsPrediction_Simple.csproj`.
- README step 3 says "Command Prompt / PowerPoint / Terminal"; this should read PowerShell.
- README says the app prints http://localhost:5000; the actual port depends on launchSettings and may differ.

10. Recommended Fixes and Enhancements

Highest priority: input validation

Treat out-of-range values as missing (null) and make the builder skip nulls rather than averaging zeros:

```
static float? Clean(float? v, float min, float max)
    => v is { } x && x >= min && x <= max ? x : null;

// Plausible human ranges
var systolic  = Clean(r.Systolic,  50, 260);   // mmHg
var diastolic = Clean(r.Diastolic, 30, 160);   // mmHg
var glucose   = Clean(r.Glucose,   20, 600);   // mg/dL
var oxygen    = Clean(r.Oxygen,    50, 100);   // SpO2 %
var weight    = Clean(r.WeightKg,   2, 350);   // kg

// In TrendFeatureBuilder: compute averages over non-null values only,
// and use the first and last NON-NULL reading for trends.
```

Then require a minimum number of valid readings per tracked vital, and return a 400 error naming the offending reading when too few remain.

Area	Recommendation
Trend units	Divide by elapsed days (from RecordedAt) so the "per day" labels are true; optionally use a least-squares slope over all readings.
Training data	Replace synthetic labels with real, labelled longitudinal data; UCI Heart Disease and Pima Diabetes are cross-sectional, so a time-series source is needed for trend features.
Model quality	Add cross-validation, compare trainers (FastTree, LightGbm) by AUC, and report a confusion matrix.
Explanations	Derive contributors from model feature weights or SHAP-style attribution so text and prediction agree.
Performance	Use PredictionEnginePool or call model.Transform on a one-row IDataView instead of building an engine per request.
Features	Add rolling z-score anomaly detection and reading-gap detection.
Persistence	Add SQLite/Postgres to store patients and readings over time.
Config	Move risk thresholds and plausibility ranges to appsettings.json.
Tests	Add unit tests for validation and feature building, including zero, null and outlier cases.

11. Limitations

- Training data is synthetic; the model has no clinical validity.
- Only one trainer and one 80/20 split are used; no cross-validation.
- Features capture only averages and first-to-last change.
- One patient per request, no persistence.

- Only one input file was tested, and it is largely unrecorded; a clean, realistic file should be tested for contrast.
- Model accuracy/AUC and sample UI output were not captured for this report. Run the app and add them (see Section 12).

12. How to Run

```
# Requires the .NET 9 SDK:  dotnet --version
cd <folder containing the .csproj>
dotnet run
# Open the URL printed in the console, click "Load sample data" then "Predict risk",
# or upload sample-patient-input.json.
```

Model accuracy and AUC are returned in the JSON response of `/api/predict` (fields `ModelAccuracy` and `ModelAuc`). Expect very high values because the labels are generated from the same features.

13. Conclusion

The pipeline works end to end in a small, readable codebase: irregular vitals become eight trend features, a logistic-regression model scores them, and the engines translate the result into guidance. The weakest links are not the model but the inputs and labels: zeros and outliers flow through unchecked, and the training labels only encode the author's own thresholds. Range validation with null-for-missing handling, correct per-day trends and a real labelled dataset are the highest-value next steps.

References

Ahmed, Z. et al. (2019). Machine Learning at Microsoft with ML.NET. KDD 2019. | Liu, D. C. and Nocedal, J. (1989). On the limited memory BFGS method for large scale optimization. Mathematical Programming 45, 503-528. | Royal College of Physicians (2017). National Early Warning Score (NEWS) 2. | Detrano, R. et al. (1989). Am. J. Cardiology 64(5), 304-310. | Smith, J. W. et al. (1988). Proc. SCAMC, 261-265. | Microsoft ML.NET documentation, learn.microsoft.com/dotnet/machine-learning.